

is taken into account, the right-side manipulations can be reduced to only $\frac{1}{6}N^3$ loop executions, and, for matrix inversion, the two methods have identical efficiencies.

Both Gaussian elimination and Gauss-Jordan elimination share the disadvantage that all right-hand sides must be known in advance. The LU decomposition method in the next section does not share that deficiency, and also has an equally small operations count, both for solution with any number of right-hand sides and for matrix inversion.

CITED REFERENCES AND FURTHER READING:

- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.3–1.
- Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), §2.1.
- Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §2.2.1.
- Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).

2.3 LU Decomposition and Its Applications

Suppose we are able to write the matrix $\mathbf{A}$ as a product of two matrices,

$$\mathbf{L} \cdot \mathbf{U} = \mathbf{A} \quad (2.3.1)$$

where $\mathbf{L}$ is *lower triangular* (has elements only on the diagonal and below) and $\mathbf{U}$ is *upper triangular* (has elements only on the diagonal and above). For the case of a 4×4 matrix $\mathbf{A}$, for example, equation (2.3.1) would look like this:

$$\begin{bmatrix} \alpha_{00} & 0 & 0 & 0 \\ \alpha_{10} & \alpha_{11} & 0 & 0 \\ \alpha_{20} & \alpha_{21} & \alpha_{22} & 0 \\ \alpha_{30} & \alpha_{31} & \alpha_{32} & \alpha_{33} \end{bmatrix} \cdot \begin{bmatrix} \beta_{00} & \beta_{01} & \beta_{02} & \beta_{03} \\ 0 & \beta_{11} & \beta_{12} & \beta_{13} \\ 0 & 0 & \beta_{22} & \beta_{23} \\ 0 & 0 & 0 & \beta_{33} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \quad (2.3.2)$$

We can use a decomposition such as (2.3.1) to solve the linear set

$$\mathbf{A} \cdot \mathbf{x} = (\mathbf{L} \cdot \mathbf{U}) \cdot \mathbf{x} = \mathbf{L} \cdot (\mathbf{U} \cdot \mathbf{x}) = \mathbf{b} \quad (2.3.3)$$

by first solving for the vector $\mathbf{y}$ such that

$$\mathbf{L} \cdot \mathbf{y} = \mathbf{b} \quad (2.3.4)$$

and then solving

$$\mathbf{U} \cdot \mathbf{x} = \mathbf{y} \quad (2.3.5)$$

What is the advantage of breaking up one linear set into two successive ones? The advantage is that the solution of a triangular set of equations is quite trivial, as

we have already seen in §2.2 (equation 2.2.4). Thus, equation (2.3.4) can be solved by *forward substitution* as follows:

$$\begin{aligned} y_0 &= \frac{b_0}{\alpha_{00}} \\ y_i &= \frac{1}{\alpha_{ii}} \left[b_i - \sum_{j=0}^{i-1} \alpha_{ij} y_j \right] \quad i = 1, 2, \dots, N-1 \end{aligned} \quad (2.3.6)$$

while (2.3.5) can then be solved by *backsubstitution* exactly as in equations (2.2.2) – (2.2.4),

$$\begin{aligned} x_{N-1} &= \frac{y_{N-1}}{\beta_{N-1,N-1}} \\ x_i &= \frac{1}{\beta_{ii}} \left[y_i - \sum_{j=i+1}^{N-1} \beta_{ij} x_j \right] \quad i = N-2, N-3, \dots, 0 \end{aligned} \quad (2.3.7)$$

Equations (2.3.6) and (2.3.7) total (for each right-hand side **b**) N^2 executions of an inner loop containing one multiply and one add. If we have N right-hand sides that are the unit column vectors (which is the case when we are inverting a matrix), then taking into account the leading zeros reduces the total execution count of (2.3.6) from $\frac{1}{2}N^3$ to $\frac{1}{6}N^3$, while (2.3.7) is unchanged at $\frac{1}{2}N^3$.

Notice that, once we have the *LU* decomposition of **A**, we can solve with as many right-hand sides as we then care to, one at a time. This is a distinct advantage over the methods of §2.1 and §2.2.

2.3.1 Performing the LU Decomposition

How then can we solve for **L** and **U**, given **A**? First, we write out the i, j th component of equation (2.3.1) or (2.3.2). That component always is a sum beginning with

$$\alpha_{i0}\beta_{0j} + \dots = a_{ij}$$

The number of terms in the sum depends, however, on whether i or j is the smaller number. We have, in fact, the three cases,

$$i < j : \quad \alpha_{i0}\beta_{0j} + \alpha_{i1}\beta_{1j} + \dots + \alpha_{ii}\beta_{ij} = a_{ij} \quad (2.3.8)$$

$$i = j : \quad \alpha_{i0}\beta_{0j} + \alpha_{i1}\beta_{1j} + \dots + \alpha_{ii}\beta_{jj} = a_{ij} \quad (2.3.9)$$

$$i > j : \quad \alpha_{i0}\beta_{0j} + \alpha_{i1}\beta_{1j} + \dots + \alpha_{ij}\beta_{jj} = a_{ij} \quad (2.3.10)$$

Equations (2.3.8) – (2.3.10) total N^2 equations for the $N^2 + N$ unknown α 's and β 's (the diagonal being represented twice). Since the number of unknowns is greater than the number of equations, we are invited to specify N of the unknowns arbitrarily and then try to solve for the others. In fact, as we shall see, it is always possible to take

$$\alpha_{ii} \equiv 1 \quad i = 0, \dots, N-1 \quad (2.3.11)$$

A surprising procedure, now, is *Crout's algorithm*, which quite trivially solves the set of $N^2 + N$ equations (2.3.8) – (2.3.11) for all the α 's and β 's by just arranging the equations in a certain order! That order is as follows:

- Set $\alpha_{ii} = 1, i = 0, \dots, N - 1$ (equation 2.3.11).
- For each $j = 0, 1, 2, \dots, N - 1$ do these two procedures: First, for $i = 0, 1, \dots, j$, use (2.3.8), (2.3.9), and (2.3.11) to solve for β_{ij} , namely

$$\beta_{ij} = a_{ij} - \sum_{k=0}^{i-1} \alpha_{ik} \beta_{kj} \quad (2.3.12)$$

(When $i = 0$ in 2.3.12 the summation term is taken to mean zero.) Second, for $i = j + 1, j + 2, \dots, N - 1$ use (2.3.10) to solve for α_{ij} , namely

$$\alpha_{ij} = \frac{1}{\beta_{jj}} \left(a_{ij} - \sum_{k=0}^{j-1} \alpha_{ik} \beta_{kj} \right) \quad (2.3.13)$$

Be sure to do both procedures before going on to the next j .

If you work through a few iterations of the above procedure, you will see that the α 's and β 's that occur on the right-hand side of equations (2.3.12) and (2.3.13) are already determined by the time they are needed. You will also see that every a_{ij} is used only once and never again. This means that the corresponding α_{ij} or β_{ij} can be stored in the location that the a used to occupy: the decomposition is “in place.” [The diagonal unity elements α_{ii} (equation 2.3.11) are not stored at all.] In brief, Crout's method fills in the combined matrix of α 's and β 's,

$$\begin{bmatrix} \beta_{00} & \beta_{01} & \beta_{02} & \beta_{03} \\ \alpha_{10} & \beta_{11} & \beta_{12} & \beta_{13} \\ \alpha_{20} & \alpha_{21} & \beta_{22} & \beta_{23} \\ \alpha_{30} & \alpha_{31} & \alpha_{32} & \beta_{33} \end{bmatrix} \quad (2.3.14)$$

by columns from left to right, and within each column from top to bottom (see Figure 2.3.1).

What about pivoting? Pivoting (i.e., selection of a salubrious pivot element for the division in equation 2.3.13) is absolutely essential for the stability of Crout's method. Only partial pivoting (interchange of rows) can be implemented efficiently. However this is enough to make the method stable. This means, incidentally, that we don't actually decompose the matrix $\mathbf{A}$ into LU form, but rather we decompose a rowwise permutation of $\mathbf{A}$. (If we keep track of what that permutation is, this decomposition is just as useful as the original one would have been.)

Pivoting is slightly subtle in Crout's algorithm. The key point to notice is that equation (2.3.12) in the case of $i = j$ (its final application) is *exactly the same* as equation (2.3.13) except for the division in the latter equation; in both cases the upper limit of the sum is $k = j - 1$ ($= i - 1$). This means that we don't have to commit ourselves as to whether the diagonal element β_{jj} is the one that happens to fall on the diagonal in the first instance, or whether one of the (undivided) α_{ij} 's below it in the column, $i = j + 1, \dots, N - 1$, is to be “promoted” to become the diagonal β . This can be decided after all the candidates in the column are in hand. As you should be able to guess by now, we will choose the largest one as the diagonal β (pivot

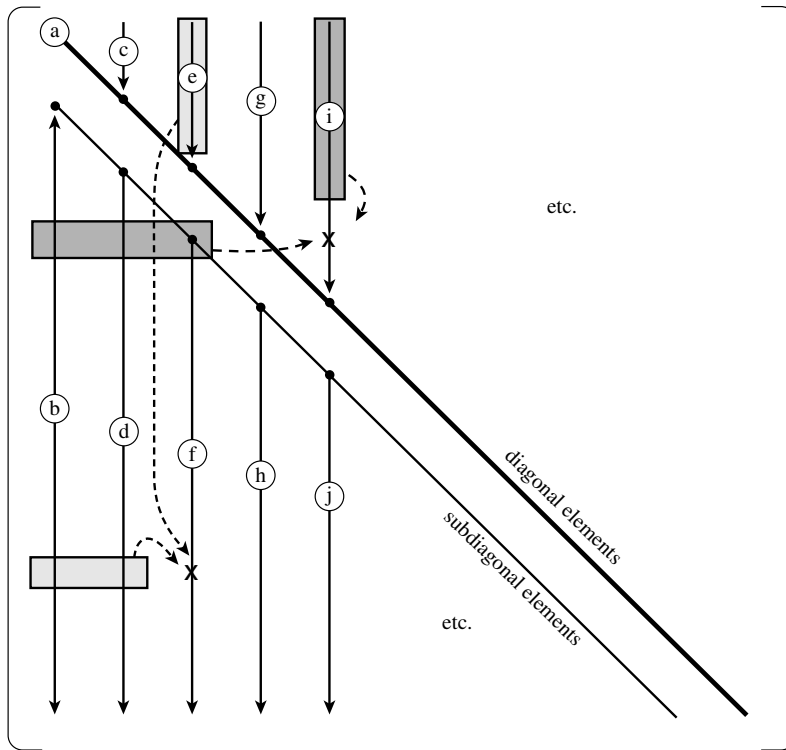


Figure 2.3.1. Crout's algorithm for LU decomposition of a matrix. Elements of the original matrix are modified in the order indicated by lowercase letters: a, b, c, etc. Shaded boxes show the previously modified elements that are used in modifying two typical elements, each indicated by an "x".

element), and then do all the divisions by that element *en masse*. This is *Crout's method with partial pivoting*. Our implementation has one additional wrinkle: It initially finds the largest element in each row, and subsequently (when it is looking for the maximal pivot element) scales the comparison *as if* we had initially scaled all the equations to make their maximum coefficient equal to unity; this is the *implicit pivoting* mentioned in §2.1.

The inner loop of the LU decomposition, equations (2.3.12) and (2.3.13), resembles the inner loop of matrix multiplication. There is a triple loop over the indices i , j , and k . There are six permutations of the order in which these loops can be done. The straightforward implementation of Crout's algorithm corresponds to the jik permutation, where the order of the indices is the order of the loops from outermost to innermost. On modern processors with a hierarchy of cache memory, and when matrices are stored by rows, the fastest execution time is usually the kij or ikj ordering. Pivoting is easier with kij ordering, so that is the implementation we use. This is called "outer product Gaussian elimination" by Golub and Van Loan [1].

LU decomposition is well suited for implementation as an object (a `class` or `struct`). The constructor performs the decomposition, and the object itself stores the result. Then, a method for forward- and backsubstitution can be called once, or many times, to solve for one or more right-hand sides. Methods for additional functionality are also easy to include. The object's declaration looks like this:

ludcmp.h

struct LUdcmp

Object for solving linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ using LU decomposition, and related functions.

```

{
    Int n;
    MatDoub lu;
    VecInt indx;
    Doub d;
    LUdcmp(MatDoub_I &a);
    void solve(VecDoub_I &b, VecDoub_O &x);
    void solve(MatDoub_I &b, MatDoub_O &x);
    void inverse(MatDoub_O &ainv);
    Doub det();
    void mprove(VecDoub_I &b, VecDoub_IO &x);
    MatDoub_I &aref;
};

```

Stores the decomposition.
Stores the permutation.
Used by det.
Constructor. Argument is the matrix $\mathbf{A}$.
Solve for a single right-hand side.
Solve for multiple right-hand sides.
Calculate matrix inverse $\mathbf{A}^{-1}$.
Return determinant of $\mathbf{A}$.
Discussed in §2.5.
Used only by mprove.

Here is the implementation of the constructor, whose argument is the input matrix that is to be LU decomposed. The input matrix is not altered; a copy is made, on which outer product Gaussian elimination is then done in-place.

ludcmp.h

LUdcmp::LUdcmp(MatDoub_I &a) : n(a.nrows()), lu(a), aref(a), indx(n) {

Given a matrix $\mathbf{a}[0..n-1][0..n-1]$, this routine replaces it by the LU decomposition of a rowwise permutation of itself. $\mathbf{a}$ is input. On output, it is arranged as in equation (2.3.14) above; $\text{indx}[0..n-1]$ is an output vector that records the row permutation effected by the partial pivoting; d is output as ± 1 depending on whether the number of row interchanges was even or odd, respectively. This routine is used in combination with `solve` to solve linear equations or invert a matrix.

```

    const Doub TINY=1.0e-40;
    Int i,imax,j,k;
    Doub big,temp;
    VecDoub vv(n);
    d=1.0;
    for (i=0;i<n;i++) {
        big=0.0;
        for (j=0;j<n;j++)
            if ((temp=abs(lu[i][j])) > big) big=temp;
        if (big == 0.0) throw("Singular matrix in LUdcmp");
        No nonzero largest element.
        vv[i]=1.0/big;
    }
    for (k=0;k<n;k++) {
        big=0.0;
        for (i=k;i<n;i++) {
            temp=vv[i]*abs(lu[i][k]);
            if (temp > big) {
                big=temp;
                imax=i;
            }
        }
        if (k != imax) {
            for (j=0;j<n;j++) {
                temp=lu[imax][j];
                lu[imax][j]=lu[k][j];
                lu[k][j]=temp;
            }
            d = -d;
            vv[imax]=vv[k];
        }
        indx[k]=imax;
        if (lu[k][k] == 0.0) lu[k][k]=TINY;
    }

```

A small number.

vv stores the implicit scaling of each row.
No row interchanges yet.
Loop over rows to get the implicit scaling information.

Save the scaling.

This is the outermost kij loop.
Initialize for the search for largest pivot element.

Is the figure of merit for the pivot better than the best so far?

Do we need to interchange rows?
Yes, do so...

...and change the parity of d .
Also interchange the scale factor.

If the pivot element is zero, the matrix is singular (at least to the precision of the algorithm). For some applications on singular matrices, it is desirable to substitute

```

TINY for zero.
for (i=k+1;i<n;i++) {
    temp=lu[i][k] /= lu[k][k]; Divide by the pivot element.
    for (j=k+1;j<n;j++) Innermost loop: reduce remaining submatrix.
        lu[i][j] -= temp*lu[k][j];
    }
}
}
}

```

Once the LUdcmp object is constructed, two functions implementing equations (2.3.6) and (2.3.7) are available for solving linear equations. The first solves a single right-hand side vector **b** for a solution vector **x**. The second simultaneously solves multiple right-hand vectors, arranged as the columns of a matrix **B**. In other words, it calculates the matrix $\mathbf{A}^{-1} \cdot \mathbf{B}$.

```
void LUdcmp::solve(VecDoub_I &b, VecDoub_O &x)
```

ludcmp.h

Solves the set of n linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ using the stored LU decomposition of **A**. **b**[0.. $n-1$] is input as the right-hand side vector **b**, while **x** returns the solution vector **x**; **b** and **x** may reference the same vector, in which case the solution overwrites the input. This routine takes into account the possibility that **b** will begin with many zero elements, so it is efficient for use in matrix inversion.

```

{
    Int i,ii=0,ip,j;
    Doub sum;
    if (b.size() != n || x.size() != n)
        throw("LUdcmp::solve bad sizes");
    for (i=0;i<n;i++) x[i] = b[i];
    for (i=0;i<n;i++) {
        ip=indx[i];
        sum=x[ip];
        x[ip]=x[i];
        if (ii != 0)
            for (j=ii-1;j<i;j++) sum -= lu[i][j]*x[j];
        else if (sum != 0.0)
            ii=i+1;
        x[i]=sum;
    }
    for (i=n-1;i>=0;i--) {
        sum=x[i];
        for (j=i+1;j<n;j++) sum -= lu[i][j]*x[j];
        x[i]=sum/lu[i][i];
    }
}

```

When **ii** is set to a positive value, it will become the index of the first nonvanishing element of **b**. We now do the forward substitution, equation (2.3.6). The only new wrinkle is to unscramble the permutation as we go.

A nonzero element was encountered, so from now on we will have to do the sums in the loop above.

Now we do the backsubstitution, equation (2.3.7).

Store a component of the solution vector **X**.

All done!

```
void LUdcmp::solve(MatDoub_I &b, MatDoub_O &x)
```

Solves m sets of n linear equations $\mathbf{A} \cdot \mathbf{X} = \mathbf{B}$ using the stored LU decomposition of **A**. The matrix **b**[0.. $n-1$][0.. $m-1$] inputs the right-hand sides, while **x**[0.. $n-1$][0.. $m-1$] returns the solution $\mathbf{A}^{-1} \cdot \mathbf{B}$. **b** and **x** may reference the same matrix, in which case the solution overwrites the input.

```

{
    int i,j,m=b.ncols();
    if (b.nrows() != n || x.nrows() != n || b.ncols() != x.ncols())
        throw("LUdcmp::solve bad sizes");
    VecDoub xx(n);
    for (j=0;j<m;j++) {
        Copy and solve each column in turn.
        for (i=0;i<n;i++) xx[i] = b[i][j];
        solve(xx,xx);
        for (i=0;i<n;i++) x[i][j] = xx[i];
    }
}

```

The LU decomposition in `LUdcmp` requires about $\frac{1}{3}N^3$ executions of the inner loops (each with one multiply and one add). This is thus the operation count for solving one (or a few) right-hand sides, and is a factor of 3 better than the Gauss-Jordan routine `gaussj` that was given in §2.1, and a factor of 1.5 better than a Gauss-Jordan routine (not given) that does not compute the inverse matrix. For inverting a matrix, the total count (including the forward- and backsubstitution as discussed following equation 2.3.7 above) is $(\frac{1}{3} + \frac{1}{6} + \frac{1}{2})N^3 = N^3$, the same as `gaussj`.

To summarize, this is the preferred way to solve the linear set of equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$:

```
const Int n = ...
MatDoub a(n,n);
VecDoub b(n),x(n);
...
LUdcmp alu(a);
alu.solve(b,x);
```

The answer will be given back in `x`. Your original matrix `a` and vector `b` are not altered. If you need to recover the storage in the object `alu`, then start a temporary scope with “{” before `alu` is declared, and end that scope with “}” when you want `alu` to be destroyed.

If you subsequently want to solve a set of equations with the same $\mathbf{A}$ but a different right-hand side $\mathbf{b}$, you repeat *only*

```
alu.solve(b,x);
```

2.3.2 Inverse of a Matrix

`LUdcmp` has a member function that gives the inverse of the matrix $\mathbf{A}$. Simply, it creates an identity matrix and then invokes the appropriate `solve` method.

`ludcmp.h`

```
void LUdcmp::inverse(MatDoub_0 &ainv)
```

Using the stored LU decomposition, return in `ainv` the matrix inverse $\mathbf{A}^{-1}$.

```
{
    Int i,j;
    ainv.resize(n,n);
    for (i=0;i<n;i++) {
        for (j=0;j<n;j++) ainv[i][j] = 0.;
        ainv[i][i] = 1.;
    }
    solve(ainv,ainv);
}
```

The matrix `ainv` will now contain the inverse of the original matrix `a`. Alternatively, there is nothing wrong with using a Gauss-Jordan routine like `gaussj` (§2.1) to invert a matrix in place, destroying the original. Both methods have practically the same operations count.

2.3.3 Determinant of a Matrix

The determinant of an LU decomposed matrix is just the product of the diagonal elements,

$$\det = \prod_{j=0}^{N-1} \beta_{jj} \quad (2.3.15)$$

We don't, recall, compute the decomposition of the original matrix, but rather a decomposition of a rowwise permutation of it. Luckily, we have kept track of whether the number of row interchanges was even or odd, so we just preface the product by the corresponding sign. (You now finally know the purpose of `d` in the `LUdcmp` structure.)

`Doub LUDcmp::det()`

`ludcmp.h`

Using the stored *LU* decomposition, return the determinant of the matrix **A**.

```
{
    Doub dd = d;
    for (Int i=0; i<n; i++) dd *= lu[i][i];
    return dd;
}
```

For a matrix of any substantial size, it is quite likely that the determinant will overflow or underflow your computer's floating-point dynamic range. In such a case you can easily add another member function that, e.g., divides by powers of ten, to keep track of the scale separately, or, e.g., accumulates the sum of logarithms of the absolute values of the factors and the sign separately.

2.3.4 Complex Systems of Equations

If your matrix **A** is real, but the right-hand side vector is complex, say **b** + *i***d**, then (i) *LU* decompose **A** in the usual way, (ii) backsubstitute **b** to get the real part of the solution vector, and (iii) backsubstitute **d** to get the imaginary part of the solution vector.

If the matrix itself is complex, so that you want to solve the system

$$(\mathbf{A} + i\mathbf{C}) \cdot (\mathbf{x} + i\mathbf{y}) = (\mathbf{b} + i\mathbf{d}) \quad (2.3.16)$$

then there are two possible ways to proceed. The best way is to rewrite `LUdcmp` with complex routines. Complex modulus substitutes for absolute value in the construction of the scaling vector `vv` and in the search for the largest pivot elements. Everything else goes through in the obvious way, with complex arithmetic used as needed.

A quick-and-dirty way to solve complex systems is to take the real and imaginary parts of (2.3.16), giving

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} - \mathbf{C} \cdot \mathbf{y} &= \mathbf{b} \\ \mathbf{C} \cdot \mathbf{x} + \mathbf{A} \cdot \mathbf{y} &= \mathbf{d} \end{aligned} \quad (2.3.17)$$

which can be written as a $2N \times 2N$ set of *real* equations,

$$\begin{pmatrix} \mathbf{A} & -\mathbf{C} \\ \mathbf{C} & \mathbf{A} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} = \begin{pmatrix} \mathbf{b} \\ \mathbf{d} \end{pmatrix} \quad (2.3.18)$$

and then solved with `LUdcmp`'s routines in their present forms. This scheme is a factor of 2 inefficient in storage, since **A** and **C** are stored twice. It is also a factor of 2 inefficient in time, since the complex multiplies in a complexified version of the routines would each use 4 real multiplies, while the solution of a $2N \times 2N$ problem involves 8 times the work of an $N \times N$ one. If you can tolerate these factor-of-two inefficiencies, then equation (2.3.18) is an easy way to proceed.

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 4.[1]